



COLEGIO DE FORMACIÓN EDUCATIVA TENAM

Doctorado en Ingeniería Aplicada

Seminario de Sistemas Inteligentes II: Robótica Móvil

Proyecto Integrador

Simulación de Navegación Autónoma en Entornos Cerrados

Mapeo, Localización y Navegación con ROS y Gazebo

Presenta:

Ismael Rufino Grajeda Marín, M.I.A.

25 de diciembre de 2025

Resumen

El presente proyecto integrador aborda la implementación completa de un sistema de navegación autónoma para un robot móvil diferencial en un entorno cerrado simulado, utilizando el framework ROS (Robot Operating System) en conjunto con el simulador Gazebo y la herramienta de visualización RViz. El proyecto se estructura en tres tareas secuenciales: (1) construcción de un mapa de rejilla de ocupación mediante SLAM con Gmapping, empleando un filtro de partículas Rao-Blackwellized; (2) localización probabilística del robot sobre el mapa generado utilizando AMCL con muestreo adaptativo KLD; y (3) navegación autónoma hacia puntos meta mediante el Navigation Stack, combinando el planificador global A* con el planificador local DWA (Dynamic Window Approach). Se detalla el diseño del robot diferencial en URDF, la cinemática asociada, la configuración paramétrica de cada paquete y el análisis de resultados esperados. El documento constituye una guía completa para la reproducción del pipeline experimental y sienta las bases para trabajos futuros en robótica móvil autónoma.

Palabras clave: Navegación autónoma, ROS, SLAM, Gmapping, AMCL, Navigation Stack, robot diferencial, simulación Gazebo.

1. Introducción

La robótica móvil autónoma ha experimentado un avance significativo en las últimas dos décadas, impulsada por la convergencia de algoritmos probabilísticos de estimación, sensores de bajo costo y frameworks de desarrollo robótico de código abierto (Siegwart et al., 2011). La capacidad de un robot para desplazarse de manera autónoma en entornos no estructurados o desconocidos requiere la resolución de tres problemas fundamentales: ¿dónde me encuentro? (localización), ¿cómo es el entorno? (mapeo) y ¿cómo llego a mi destino? (planificación de movimiento) (Thrun et al., 2005). Estos problemas, conocidos colectivamente como SLAM (Simultaneous Localization and Mapping) en el caso del mapeo con localización simultánea, constituyen el núcleo de la navegación autónoma.

El Robot Operating System (ROS) se ha consolidado como el estándar *de facto* en investigación y desarrollo robótico, proporcionando una infraestructura de comunicación entre nodos, herramientas de visualización y una extensa colección de paquetes para navegación,

percepción y control (Quigley et al., 2009). Complementariamente, Gazebo ofrece un entorno de simulación tridimensional con física realista que permite validar algoritmos sin los costos y riesgos asociados a la experimentación con hardware real (Koenig and Howard, 2004).

Este proyecto integrador tiene como objetivo implementar un pipeline completo de navegación autónoma para un robot móvil diferencial en un entorno cerrado simulado (almacén), abarcando desde el diseño del modelo cinemático hasta la ejecución de navegación basada en costmaps. Las tres tareas que conforman el proyecto —mapeo con Gmapping, localización con AMCL y navegación con el Navigation Stack— se ejecutan de manera secuencial, compartiendo un mismo mapa como elemento unificador (Marder-Eppstein et al., 2010).

El documento se organiza de la siguiente manera: la Sección 2 constituye el núcleo del trabajo, presentando en la Subsección 2.1 el diseño del robot y su cinemática, en la Subsección 2.2 la tarea de mapeo con Gmapping, en la Subsección 2.3 la localización con AMCL, y en la Subsección 2.4 la navegación con el Navigation Stack. La Sección 3 presenta el diagrama de arquitectura del sistema ROS completo. Finalmente, la Sección 4 expone las conclusiones y el trabajo futuro.

2. Desarrollo

2.1. Definición del Problema y Diseño del Robot

2.1.1. Planteamiento del Problema

Se plantea el problema de navegación autónoma para un robot móvil en un entorno cerrado desconocido, específicamente un almacén simulado con paredes, pasillos y obstáculos estáticos. El robot debe ser capaz de: (1) explorar el entorno y construir un mapa de rejilla de ocupación sin conocimiento *a priori* del mismo, (2) localizarse dentro del mapa generado a partir de observaciones sensoriales, y (3) navegar de manera autónoma hacia puntos objetivo definidos por el usuario, evadiendo obstáculos en el camino.

La elección de un robot de tracción diferencial frente a alternativas como la configuración Ackerman (vehicular) o la omnidireccional responde a criterios técnicos y de adecuación al entorno. El robot diferencial ofrece menor complejidad mecánica (dos motores

independientes), control cinemático simple basado en velocidades lineales y angulares, y alta maniobrabilidad en espacios reducidos, características ideales para navegación en interiores (Siegwart et al., 2011). A diferencia del robot Ackerman, el diferencial permite giros en el lugar (*zero-radius turn*), lo que simplifica las maniobras en pasillos estrechos. En comparación con robots omnidireccionales, el diferencial tiene menor costo computacional de control y simplicidad mecánica, aunque presenta limitaciones en movimiento lateral.

2.1.2. Cinemática del Robot Diferencial

El modelo cinemático del robot diferencial establece la relación entre las velocidades angulares de las ruedas motrices y la velocidad lineal y angular del centro del robot (Siegwart et al., 2011). Para un robot con radio de ruedas r y distancia entre ruedas (baseline) L , la cinemática directa se expresa mediante las ecuaciones (1) y (2):

$$v = \frac{(\omega_r + \omega_l) \cdot r}{2} \quad (1)$$

$$\omega = \frac{(\omega_r - \omega_l) \cdot r}{L} \quad (2)$$

donde ω_r y ω_l son las velocidades angulares de las ruedas derecha e izquierda respectivamente (en rad/s). La formulación matricial del modelo se presenta en (3):

$$\begin{bmatrix} v \\ \omega \end{bmatrix} = \begin{bmatrix} \frac{r}{2} & \frac{r}{2} \\ \frac{r}{L} & -\frac{r}{L} \end{bmatrix} \begin{bmatrix} \omega_r \\ \omega_l \end{bmatrix} \quad (3)$$

La cinemática inversa, que permite obtener las velocidades de las ruedas a partir de la velocidad deseada del robot, se define en (4):

$$\omega_r = \frac{2v + \omega L}{2r}, \quad \omega_l = \frac{2v - \omega L}{2r} \quad (4)$$

El robot diseñado utiliza los parámetros físicos listados en la Tabla 1, que se encuentran dentro de los rangos típicos para robots móviles de interior.

Tabla 1: Parámetros físicos del robot diferencial.

Parámetro	Valor	Unidad	Rango típico
Masa del chasis (m)	3.0	kg	2–5
Radio de ruedas (r)	0.075	m	0.05–0.10
Baseline (L)	0.35	m	0.20–0.35
Altura del LiDAR	0.08	m (sobre base)	–
Distancia LiDAR al centro	0.15	m (adelante)	–
Rozamiento (μ)	1.0	–	–
Coef. de amortiguación	0.1	N·s/m	–

Las limitaciones inherentes al diseño diferencial se resumen en la Tabla 2.

Tabla 2: Limitaciones del robot diferencial.

Limitación	Implicación
Sin movimiento lateral	Requiere maniobras de tres puntos para estacionamiento lateral
Deslizamiento en curvas cerradas	Error de odometría acumulativo que requiere corrección probabilística
Singularidad en $v = 0$	Imposibilidad de girar sobre el eje con precisión infinitesimal

2.1.3. Modelo URDF en Simulación

El modelo unificado del robot se define en formato URDF (Unified Robot Description Format), compatible con ROS y Gazebo. La estructura comprende cinco *links* y cuatro *joints*, organizados según la especificación de la Tabla 3.

Tabla 3: Especificación de *links* del modelo URDF.

Link	Geometría	Dimensiones (m)	Masa (kg)
base_link	Caja (box)	$0,40 \times 0,30 \times 0,15$	3.0
left_wheel_link	Cilindro	$r = 0,075, h = 0,04$	0.3
right_wheel_link	Cilindro	$r = 0,075, h = 0,04$	0.3
caster_wheel_link	Esfera	$r = 0,025$	0.05
laser_frame	Virtual	—	—

Las conexiones entre *links* se establecen mediante *joints* de tipo **revolute** para las ruedas motrices y **fixed** para los elementos de soporte y sensor, tal como se detalla en la Tabla 4.

Tabla 4: Especificación de *joints* del modelo URDF.

Joint	Tipo	Parent → Child	Origen (xyz)
base_to_left_wheel	revolute	base_link → left_wheel_link	(0, 0, 175, 0)
base_to_right_wheel	revolute	base_link → right_wheel_link	(0, -0, 175, 0)
base_to_caster	fixed	base_link → caster_wheel_link	(-0, 15, 0, -0, 04)
base_to_laser	fixed	base_link → laser_frame	(0, 15, 0, 0, 08)

Para la simulación en Gazebo, se configuran dos *transmissions* que conectan los *joints* rotacionales con la interfaz de velocidad del simulador. El fragmento URDF correspondiente a la transmisión de la rueda izquierda se muestra en el Listado 1. Adicionalmente, se emplea el plugin `libgazebo_ros_diff_drive.so` para el control diferencial y `libgazebo_ros_laser.so` para el sensor LiDAR.

Listing 1: Transmisión URDF para rueda izquierda.

```

1 <transmission name="trans_left_wheel">
2   <type>transmission_interface/SimpleTransmission</type>
3   <joint name="base_to_left_wheel">
4     <hardwareInterface>VelocityJointInterface</hardwareInterface>
5   </joint>
6   <actuator name="motor_left">
7     <hardwareInterface>VelocityJointInterface</hardwareInterface>
8     <mechanicalReduction>1</mechanicalReduction>

```

```

9   </actuator>
10 </transmission>

```

2.1.4. Sensor LiDAR: Especificaciones y Configuración

El sensor LiDAR simulado en Gazebo corresponde a un láser 2D de rayos, con especificaciones técnicas que emulan sensores comerciales como el Hokuyo URG-04LX o el SICK LMS1xx (Siegwart et al., 2011). La Tabla 5 resume los parámetros de configuración.

Tabla 5: Especificaciones técnicas del sensor LiDAR simulado.

Parámetro	Valor	Equivalente real
Tipo	Láser 2D de rayos	Hokuyo URG-04LX / SICK LMS1xx
Rango mínimo	0.10 m	0.1 m
Rango máximo	10.0 m	5.6 m (Hokuyo) / 20 m (SICK)
Ángulo de apertura	180° (configurable a 270°)	240° (Hokuyo) / 270° (SICK)
Resolución angular	1° (360 muestras)	0.36° (Hokuyo) / 0.5° (SICK)
Frecuencia	20 Hz	10 Hz (Hokuyo) / 25 Hz (SICK)
Ruido	Gaussiano $\sigma = 0,01$ m	Idealización en simulación
Tópico ROS	/scan	—
Frame	laser_frame	—

La configuración del LiDAR en Gazebo se realiza mediante el plugin `libgazebo_ros_laser.so`, especificando el ángulo de barrido, la resolución, el rango y el modelo de ruido gaussiano, como se ilustra en el Listado 2.

Listing 2: Plugin LiDAR en Gazebo.

```

1 <gazebo reference="laser_frame">
2   <sensor type="ray" name="head_laser">
3     <pose>0 0 0 0 0 0</pose>
4     <update_rate>20</update_rate>
5     <ray>
6       <scan>
7         <horizontal>
8           <samples>360</samples>
9           <resolution>1.0</resolution>

```

```

10         <min_angle>-1.5708</min_angle>
11         <max_angle>1.5708</max_angle>
12     </horizontal>
13 </scan>
14 <range>
15     <min>0.10</min>
16     <max>10.0</max>
17     <resolution>0.01</resolution>
18 </range>
19 <noise>
20     <type>gaussian</type>
21     <mean>0.0</mean>
22     <stddev>0.01</stddev>
23 </noise>
24 </ray>
25 <plugin name="laser_node" filename="libgazebo_ros_laser.so">
26     <topicName>scan</topicName>
27     <frameName>laser_frame</frameName>
28 </plugin>
29 </sensor>
30 </gazebo>

```

2.2. Tarea 1: Mapeo con Gmapping

2.2.1. Fundamento Teórico: SLAM Basado en Partículas

El problema de SLAM (Simultaneous Localization and Mapping) consiste en estimar simultáneamente la trayectoria del robot y el mapa del entorno a partir de observaciones sensoriales y comandos de control (Thrun et al., 2005). Formalmente, se busca estimar la distribución de probabilidad conjunta $p(x_{1:t}, m \mid z_{1:t}, u_{1:t})$, donde $x_{1:t}$ es la trayectoria del robot, m es el mapa, $z_{1:t}$ son las observaciones y $u_{1:t}$ los comandos de control.

Gmapping implementa el enfoque Rao-Blackwellized Particle Filter (RBPF) para SLAM (Grisetti et al., 2007). La descomposición Rao-Blackwellizada factoriza el problema como:

$$p(x_{1:t}, m \mid z_{1:t}, u_{1:t}) = p(m \mid x_{1:t}, z_{1:t}) \cdot p(x_{1:t} \mid z_{1:t}, u_{1:t}) \quad (5)$$

En esta formulación, cada partícula representa una trayectoria hipotética del robot y mantiene su propio mapa de rejilla de ocupación condicionado a dicha trayectoria (Montemerlo and Thrun, 2007). El algoritmo procede mediante los siguientes pasos:

1. **Muestreo:** Se genera una nueva generación de partículas a partir de la distribución de importancia, que combina el modelo de movimiento del robot con la observación más reciente.
2. **Actualización del mapa:** Para cada partícula, se actualiza el mapa de rejilla de ocupación basado en la trayectoria y las observaciones.
3. **Re-muestreo adaptativo:** Cuando el número efectivo de partículas N_{eff} cae por debajo de un umbral (típicamente $N_{\text{eff}} < N/2$), se realiza un re-muestreo para eliminar partículas con baja probabilidad y duplicar aquellas con alta probabilidad.

El modelo de probabilidad del escaneo LiDAR empleado por Gmapping asume que cada medición de rango es independiente, modelando la probabilidad $p(z_t \mid x_t, m)$ como una combinación de componentes: acierto (distribución gaussiana alrededor del rango esperado), mediciones cortas (distribución exponencial), mediciones máximas y mediciones aleatorias (Thrun et al., 2005).

2.2.2. Configuración del Paquete Gmapping

La configuración del nodo `slam_gmapping` se realiza mediante el archivo de parámetros YAML especificado en la Tabla 6. Se han seleccionado 22 parámetros que abarcan las cuatro categorías principales del algoritmo: filtro de partículas, resolución del mapa, modelo de odometría, umbrales de actualización y modelo de sensor.

Tabla 6: Parámetros de configuración de Gmapping.

Parámetro	Valor	Categoría
particles	80	Filtro de partículas
delta	0.05 m/celda	Resolución del mapa
xmin, ymin	-10,0, -10,0	Límites del mapa
xmax, ymax	10,0, 10,0	Límites del mapa
srr	0.01	Ruido odometría (rot/rot)
srt	0.02	Ruido odometría (rot/tras)
str	0.01	Ruido odometría (tras/rot)
stt	0.02	Ruido odometría (tras/tras)
linearUpdate	1.0 m	Umbral de actualización lineal
angularUpdate	0.5 rad	Umbral de actualización angular
temporalUpdate	3.0 s	Actualización forzada temporal
resampleThreshold	0.5	Umbral de re-muestreo
maxUrange	8.0 m	Rango máx. para el mapa
sigma	0.05 m	Desv. estándar del escaneo
kernelSize	1	Tamaño del kernel de búsqueda
lstep	0.05	Paso lineal para correlación
astep	0.05	Paso angular para correlación
iterations	5	Iteraciones de scan-matching
lsigma	0.075	Sigma para likelihood
ogain	3.0	Ganancia para re-muestreo
llsamplerange / llsamplestep	0.01 / 0.01	Muestreo likelihood (lineal)
lasamplerange / lasamplestep	0.005 / 0.005	Muestreo likelihood (angular)

La justificación de los parámetros más críticos se presenta en la Tabla 7.

Tabla 7: Justificación de parámetros clave de Gmapping.

Parámetro	Fundamento
<code>particles=80</code>	Suficiente para entorno pequeño ($\sim 10 \times 10$ m). Cada partícula tiene su propio mapa; 80 partículas dan buena cobertura sin saturar CPU.
<code>delta=0.05</code>	Resolución estándar para interiores. Un ambiente de 10×10 m produce un mapa de 200×200 celdas.
<code>linearUpdate=1.0</code>	Actualiza el mapa cada metro recorrido. Reduce computación comparado con 0.5 m.
<code>angularUpdate=0.5</code>	Actualiza al girar $\sim 28,6^\circ$. Suficiente para corregir deriva angular.
<code>srr=0.01 a 0.02</code>	Modelo de odometría conservador (poco ruido). Ideal para simulación donde la odometría es más precisa que en hardware real.

2.2.3. Procedimiento Experimental

El procedimiento para la ejecución de la tarea de mapeo sigue el pipeline secuencial definido en el ADR-008 (Marder-Eppstein et al., 2010) y se describe a continuación:

1. **Lanzamiento del entorno:** Se ejecuta `roslaunch robot_gazebo robot.launch`, que inicia Gazebo con el entorno de almacén y el robot URDF.
2. **Lanzamiento de Gmapping:** Se ejecuta el nodo `slam_gmapping` con los parámetros de la Tabla 6. Esto puede realizarse mediante un *launch file* que cargue el archivo YAML correspondiente.
3. **Teleoperación:** Se utiliza `teleop_twist_keyboard` para teleoperar el robot cubriendo sistemáticamente todo el entorno, asegurando que cada área sea observada desde múltiples ángulos para facilitar el cierre de bucles.
4. **Guardado del mapa:** Una vez completada la exploración, se ejecuta `roslaunch map_server`

`map_saver -f ~/maps/mapa_final`, que genera dos archivos: `mapa_final.pgm` (imagen en formato PGM con probabilidades de ocupación) y `mapa_final.yaml` (metadatos del mapa).

El formato de salida del mapa se especifica en el archivo YAML generado (Listado 3).

Listing 3: Archivo de metadatos del mapa generado (`mapa_final.yaml`).

```

1 image: mapa_final.pgm
2 resolution: 0.050000
3 origin: [-10.0, -10.0, 0.0]
4 negate: 0
5 occupied_thresh: 0.65
6 free_thresh: 0.196

```

2.2.4. Resultados Esperados

Se espera que el mapa resultante presente bordes bien definidos correspondientes a las paredes del entorno, con una cobertura superior al 90 % del área explorable. El cierre de bucles (*loop closure*) debe manifestarse cuando el robot retorna a una posición previamente visitada, corrigiendo la deriva acumulada de la odometría. Cuantitativamente, se espera un tiempo de exploración de 2–5 minutos para un entorno de 10×10 m, generando un mapa de aproximadamente 200×200 celdas con una resolución de 5 cm.

En comparación con alternativas como Hector SLAM (que no requiere odometría pero demanda LiDAR de alta frecuencia) o EKF-SLAM (que mantiene una representación de landmarks en lugar de rejilla), Gmapping ofrece el mejor balance para entornos interiores con odometría disponible (Grisetti et al., 2007).

2.3. Tarea 2: Localización con AMCL

2.3.1. Fundamento Teórico: Filtro de Partículas Adaptativo

La localización probabilística aborda el problema de estimar la pose del robot $x_t = (x, y, \theta)$ dado un mapa conocido m , las observaciones $z_{1:t}$ y los comandos de control $u_{1:t}$ (Thrun et al., 2005). AMCL (Adaptive Monte Carlo Localization) implementa un filtro de partículas

con muestreo adaptativo basado en KLD (Kullback-Leibler Divergence), introducido por Fox (2003).

El algoritmo KLD-sampling ajusta dinámicamente el número de partículas n según la distribución de probabilidad estimada:

$$n = \frac{1}{2\epsilon} \chi_{k-1, 1-\delta}^2 \quad (6)$$

donde ϵ es el error máximo permitido (parámetro `kld_err`), $1 - \delta$ es la probabilidad de que el error esté dentro de ϵ , y k es el número de bins (intervalos discretos del espacio de estados) que contienen al menos una partícula. Esta expresión garantiza que la divergencia de Kullback-Leibler entre la distribución de partículas y la distribución real sea menor que ϵ con probabilidad $1 - \delta$ (Fox, 2003).

Cuando el robot tiene alta incertidumbre (localización global o recuperación de secuestro), k es grande y n se aproxima a `max_particles` (5000). Cuando converge, k se reduce y n tiende a `min_particles` (500), optimizando el costo computacional.

El modelo de sensor empleado es `likelihood_field`, que evalúa la verosimilitud de una medición observada z_t^k contra el campo de distancia precomputado del mapa (Thrun et al., 2005):

$$p(z_t^k | x_t, m) = z_{\text{hit}} \cdot \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{d^2}{2\sigma^2}\right) + \frac{z_{\text{rand}}}{z_{\text{max}}} \quad (7)$$

donde d es la distancia desde el punto de impacto del rayo hasta el obstáculo más cercano en el mapa, σ es la desviación estándar (`laser_sigma_hit`) y z_{hit} , z_{rand} son los pesos de los componentes de acierto y aleatorio respectivamente.

2.3.2. Configuración del Paquete AMCL

La configuración completa de AMCL comprende 24 parámetros organizados en cinco categorías, tal como se detalla en la Tabla 8.

Tabla 8: Parámetros de configuración de AMCL.

Parámetro	Valor	Categoría
min_particles	500	Filtro de partículas
max_particles	5000	Filtro de partículas
kld_err	0.01	KLD sampling (ϵ)
kld_z	0.99	KLD sampling (δ)
update_min_d	0.25 m	Re-muestreo
update_min_a	0.2 rad	Re-muestreo
resample_interval	1	Re-muestreo
odom_model_type	diff	Modelo de odometría
odom_alpha1	0.2	Ruido odometría
odom_alpha2	0.2	Ruido odometría
odom_alpha3	0.2	Ruido odometría
odom_alpha4	0.2	Ruido odometría
odom_alpha5	0.1	Ruido odometría
laser_model_type	likelihood_field	Modelo de sensor
laser_z_hit	0.95	Sensor: peso de acierto
laser_z_rand	0.05	Sensor: peso aleatorio
laser_sigma_hit	0.2 m	Sensor: desviación estándar
laser_lambda_short	0.1	Sensor: tasa corta
laser_max_beams	60	Sensor: muestreo de beams
initial_cov_xx/yy	0.5	Covarianza inicial
initial_cov_aa	0.5	Covarianza inicial
recovery_alpha_slow	0.001	Recuperación de fallos
recovery_alpha_fast	0.1	Recuperación de fallos
save_pose_rate	0.5 Hz	Persistencia

La justificación de los parámetros más relevantes se presenta en la Tabla 9.

Tabla 9: Justificación de parámetros clave de AMCL.

Parámetro	Fundamento
<code>min_particles=500</code>	KLD-sampling reduce partículas al converger. 500 es mínimo para mantener diversidad suficiente.
<code>max_particles=5000</code>	Para localización global se necesitan ~ 5000 partículas. Al converger, KLD reduce a ~ 500 .
<code>laser_model_type=likelihood_field</code>	En simulación, el campo de verosimilitud es más estable que <code>beam</code> porque no hay discrepancias geométricas reales (Fox, 2003).
<code>laser_z_hit=0.95</code>	Alta confianza en el sensor (simulación ideal). En hardware real se reduciría a ~ 0.8 .
<code>update_min_d=0.25</code>	Re-muestreo cada 25 cm. Balance entre precisión y costo computacional.

2.3.3. Procedimiento Experimental

El procedimiento para la localización se ejecuta después de haber completado la Tarea 1 y contar con el mapa guardado:

1. **Lanzamiento del entorno:** Se ejecuta `roslaunch robot_gazebo robot.launch` con el robot ubicado en una pose conocida dentro del entorno.
2. **Carga del mapa:** Se ejecuta `roslaunch map_server map_server ~/maps/mapa_final.yaml`, que publica el mapa en el tópico `/map`.
3. **Lanzamiento de AMCL:** Se ejecuta el nodo `amcl` con los parámetros de la Tabla 8, ya sea directamente o mediante un *launch file*.
4. **Inicialización de la pose:** En RViz, se utiliza la herramienta “2D Pose Estimate” para publicar la pose inicial estimada en el tópico `/initialpose`. AMCL recibe esta estimación y distribuye las partículas alrededor de la pose proporcionada.

5. **Convergencia:** Se observa la nube de partículas (`/particlecloud`) en RViz. A medida que el robot se desplaza, las partículas convergen hacia la pose real, indicando que la localización es exitosa.

2.3.4. Resultados Esperados

Se espera que el filtro de partículas converja en menos de 10 segundos cuando se proporciona una pose inicial aproximada (localización local), y en menos de 30 segundos para localización global sin pose inicial. La nube de partículas debe visualizarse como un grupo denso alrededor de la pose real del robot, con una covarianza que refleje la incertidumbre del sistema. La robustez del algoritmo ante errores de odometría se manifiesta en la capacidad de mantener la localización incluso cuando el robot experimenta deslizamiento simulado.

2.4. Tarea 3: Navegación con Navigation Stack

2.4.1. Fundamento Teórico: Algoritmo A*

El planificador global del Navigation Stack utiliza el algoritmo A* para encontrar la trayectoria óptima desde la pose actual del robot hasta el punto objetivo, sobre el mapa de rejilla de ocupación (Hart et al., 1968). A* combina el costo real del camino recorrido $g(n)$ con una heurística estimada $h(n)$ para guiar la búsqueda:

$$f(n) = g(n) + h(n) \quad (8)$$

donde n es un nodo de la rejilla, $g(n)$ es el costo acumulado desde el inicio hasta n , y $h(n)$ es la distancia euclidiana desde n hasta la meta, que constituye una heurística admisible (no sobrestima el costo real). La optimalidad de A* está garantizada cuando la heurística es admisible y consistente (Hart et al., 1968). La configuración utilizada para el planificador global se detalla en la Tabla 10.

Tabla 10: Configuración del planificador global (NavFn / A*).

Parámetro	Valor
allow_unknown	false
default_tolerance	0.2 m
use_dijkstra	false (usa A*)
use_quadratic	true
use_grid_path	false (trayectoria suavizada)
lethal_cost	100
neutral_cost	50
cost_factor	3.0

2.4.2. Fundamento Teórico: Dynamic Window Approach (DWA)

El planificador local implementa el Dynamic Window Approach (DWA), introducido por Fox et al. (1997), que genera comandos de velocidad (v, ω) considerando únicamente las velocidades alcanzables dentro de un intervalo de tiempo t (ventana dinámica) dado el modelo de aceleración del robot.

El espacio de búsqueda se restringe a velocidades que satisfacen tres condiciones: (1) **velocidades alcanzables** dentro de la ventana dinámica dadas las aceleraciones máximas; (2) **velocidades admisibles** que permitan detenerse antes de colisionar; y (3) **velocidades permitidas** dentro de los límites establecidos. Para cada par (v, ω) muestreado, se simula una trayectoria durante un horizonte temporal Δt y se evalúa mediante la función objetivo:

$$G(v, \omega) = \alpha \cdot \text{path_dist}(v, \omega) + \beta \cdot \text{goal_dist}(v, \omega) + \gamma \cdot \text{occ_dist}(v, \omega) \quad (9)$$

donde:

- $\text{path_dist}(v, \omega)$: distancia a la trayectoria global más cercana (peso $\alpha = 32$).
- $\text{goal_dist}(v, \omega)$: distancia euclidiana al punto meta (peso $\beta = 24$).
- $\text{occ_dist}(v, \omega)$: distancia al obstáculo más cercano (peso $\gamma = 0,02$).

La configuración completa del planificador local DWA se presenta en la Tabla 11.

Tabla 11: Parámetros del planificador local DWA.

Parámetro	Valor	Categoría
max_vel_x	0.55 m/s	Límites de velocidad
min_vel_x	-0,10 m/s	Límites de velocidad
max_vel_theta	1.0 rad/s	Límites de velocidad
min_in_place_vel_theta	0.5 rad/s	Límites de velocidad
acc_lim_x	2.5 m/s ²	Aceleraciones
acc_lim_theta	3.2 rad/s ²	Aceleraciones
sim_time	1.5 s	Ventana de simulación
vx_samples	6	Muestreo de velocidades
vtheta_samples	20	Muestreo de velocidades
path_distance_bias (α)	32.0	Función objetivo
goal_distance_bias (β)	24.0	Función objetivo
occdist_scale (γ)	0.02	Función objetivo
xy_goal_tolerance	0.10 m	Tolerancias
yaw_goal_tolerance	0.05 rad	Tolerancias
controller_frequency	10.0 Hz	Controlador

2.4.3. Configuración de Costmaps

El sistema de costmaps gestiona la representación de obstáculos para la planificación. Se emplean dos costmaps: uno global (mapa completo) y uno local (ventana centrada en el robot). La configuración común a ambos se define en la Tabla 12.

Tabla 12: Parámetros comunes de costmaps.

Parámetro	Valor
obstacle_range	3.5 m
raytrace_range	4.0 m
inflation_radius	0.40 m
cost_scaling_factor	5.0
observation_sources	laser_scan_sensor
laser_scan_sensor.topic	/scan
expected_update_rate	20.0 Hz

Las configuraciones específicas de cada costmap se detallan en la Tabla 13.

Tabla 13: Configuración diferencial de costmaps global y local.

Parámetro	Global	Local
<code>global_frame</code>	<code>map</code>	<code>odom</code>
<code>update_frequency</code>	1.0 Hz	5.0 Hz
<code>publish_frequency</code>	0.5 Hz	2.0 Hz
<code>width × height</code>	20 × 20 m	3 × 3 m
<code>resolution</code>	0.05 m	0.05 m
<code>static_map</code>	<code>true</code>	<code>false</code>
<code>rolling_window</code>	<code>false</code>	<code>true</code>

El radio de inflado (`inflation_radius=0.40` m) crea una zona de seguridad alrededor de los obstáculos, penalizando trayectorias que se aproximen demasiado. El factor de escala de costo (`cost_scaling_factor=5.0`) determina la rapidez con que decae la penalización: a 40 cm del obstáculo, el costo adicional es prácticamente nulo.

2.4.4. Procedimiento Experimental

El procedimiento para la navegación integra los componentes de las tareas anteriores:

1. **Inicialización:** Se lanza Gazebo con el entorno y el robot (`robot.launch`), se carga el mapa (`map_server`) y se inicia AMCL con localización convergida.
2. **Lanzamiento de move_base:** Se ejecuta el nodo `move_base` con los archivos de configuración de costmaps y planificadores (global A* + local DWA).
3. **Envío de goal:** En RViz, se utiliza la herramienta “2D Nav Goal” para especificar un punto destino. El planificador global calcula la trayectoria óptima, y el planificador local la ejecuta generando comandos de velocidad en `/cmd_vel`.
4. **Ejecución y monitoreo:** Se observa en RViz la trayectoria planificada (global) y la trayectoria ejecutada (local). El robot debe alcanzar el objetivo evadiendo obstáculos, y el sistema reporta el estado de la navegación en `/move_base/status`.

2.4.5. Resultados Esperados

Se espera que el planificador global A* genere una trayectoria óptima en menos de 100 ms para mapas de 200×200 celdas. El planificador local DWA debe ejecutar la trayectoria con suavidad, manteniendo velocidades dentro de los límites establecidos ($v_{\max} = 0,55$ m/s, $\omega_{\max} = 1,0$ rad/s) y evadiendo obstáculos con una distancia de seguridad de 40 cm. La eficiencia de la navegación se mide como la relación entre la distancia recorrida y la distancia euclidiana ideal, esperándose valores cercanos a 1.0 para rutas sin obstáculos y menores pero aceptables para rutas con obstáculos.

3. Diagrama de Arquitectura del Sistema

3.1. Nodos y Tópicos ROS

La arquitectura completa del sistema integra los nodos correspondientes a las tres tareas, conectados mediante los tópicos y servicios especificados en la Tabla 14. El diagrama de flujo de datos (Figura 1) ilustra las interconexiones entre los componentes.

```

1 flowchart TB
2     subgraph GAZEBO["[Gaz] Gazebo"]
3         GAZ[gazebo-server]
4         DIFF[<b>diff_drive_plugin</b><br/>libgazebo_ros_diff_drive.so]
5         LASER[<b>laser_plugin</b><br/>libgazebo_ros_laser.so]
6         ENV[Entorno .world]
7     end
8
9     subgraph CORE["[ROS] Core ROS"]
10        RSP[robot_state_publisher<br/>URDF->TF]
11        TF[/tf, /tf_static<br/>Transformaciones]
12    end
13
14    subgraph T1["[T1] SLAM"]
15        GMAP[slam_gmapping<br/>Rao-Blackwellized PF]
16        MAP_SAVER[map_saver<br/>map_server]
17        MAP_FILE[(map.pgm + map.yaml)]
18    end
19
20    subgraph T2["[T2] Localizacion"]
21        MAP_SERVER[map_server<br/>carga mapa]
22        AMCL[amcl<br/>KLD-Adaptive PF]
23        PARTICLE[/particlecloud<br/>PoseArray]
24        AMCL_POSE[/amcl_pose<br/>PoseWithCovariance]
25    end
26
27    subgraph T3["[T3] Navegacion"]
28        MV[move_base]
29        subgraph GP[Global Planner]
30            NAVFN[navfn / A*<br/>global_planner]
31            GCOSTMAP[global_costmap<br/>mapa completo]
32        end
33        subgraph LP[Local Planner]
34            DWA[dwa_local_planner<br/>Dynamic Window]
35            LCOSTMAP[local_costmap<br/>ventana 3x3 m]
36        end
37    end
38
39    subgraph UI["[UI] Interfaz"]
40        RVZ[RViz]
41        TELEOP[teleop_twist_keyboard]
42        GOAL_UI[2D Nav Goal]
43    end

```

Tabla 14: Tópicos, servicios y acciones del sistema ROS.

Tópico	Tipo	Publicador / Suscriptor
/scan	sensor_msgs/LaserScan	Gazebo → gmapping, amcl, move.L
/tf	tf/tfMessage	RSP, diff_drive → gmapping, amcl,
/tf_static	tf/tfMessage	RSP → gmapping, amcl, move.bas
/odom	nav_msgs/Odometry	diff_drive → amcl, rviz
/cmd_vel	geometry_msgs/Twist	teleop/move.base → diff_drive
/map	nav_msgs/OccupancyGrid	gmapping/map_server → amcl, mo
/initialpose	geometry_msgs/PoseWithCovarianceStamped	RViz → amcl
/amcl_pose	geometry_msgs/PoseWithCovarianceStamped	amcl → rviz
/particlecloud	geometry_msgs/PoseArray	amcl → rviz
/move_base_simple/goal	geometry_msgs/PoseStamped	RViz → move_base
/move_base/status	actionlib_msgs/GoalStatusArray	move_base → rviz

3.2. Árbol de Transformaciones (TF)

El sistema de transformaciones (TF tree) establece la jerarquía de frames que conecta el mapa global con los componentes del robot. La estructura se define en la Tabla 15.

Tabla 15: Árbol de transformaciones (TF tree).

Frame Padre	Frame Hijo	Publicador	Tipo
map	odom	amcl	Variable
odom	base_footprint	robot_state_publisher + diff_drive	Variable
base_footprint	base_link	robot_state_publisher	Fixed ($z = 0,075$ m)
base_link	laser_frame	robot_state_publisher	Fixed ($x = 0,15$, $z = 0$)
base_link	left_wheel_link	robot_state_publisher	Variable (revolute)
base_link	right_wheel_link	robot_state_publisher	Variable (revolute)
base_link	caster_wheel_link	robot_state_publisher	Fixed

La transformación $\text{map} \rightarrow \text{odom}$ es publicada por AMCL y representa la corrección global de la deriva de odometría. El frame `base_footprint` es la proyección de `base_link` al suelo ($z = 0$), simplificando la representación bidimensional del robot (Marder-Eppstein et al., 2010).

4. Conclusiones

En el presente proyecto integrador se ha implementado un pipeline completo de navegación autónoma para un robot móvil diferencial en entorno simulado, abarcando las tres tareas fundamentales de la robótica móvil: mapeo, localización y navegación. Se diseñó un modelo URDF del robot con parámetros físicos realistas y se configuró un sensor LiDAR 2D en Gazebo, estableciendo las bases para la experimentación simulada.

La Tarea 1 (mapeo con Gmapping) demuestra la efectividad del filtro de partículas Rao-Blackwellized para la construcción de mapas de rejilla de ocupación en entornos interiores, con una configuración de 80 partículas y resolución de 5 cm que balancea precisión y costo computacional. La Tarea 2 (localización con AMCL) evidencia la robustez del KLD-sampling adaptativo, que ajusta dinámicamente el número de partículas entre 500 y 5000 según la incertidumbre del sistema, utilizando el modelo de sensor `likelihood_field` para una convergencia estable en simulación.

La Tarea 3 (navegación con Navigation Stack) integra los componentes anteriores, combinando el planificador global A* con el planificador local DWA para generar trayectorias óptimas y ejecutarlas con evasión de obstáculos. La configuración paramétrica detallada en este documento, incluyendo los pesos $\alpha = 32$, $\beta = 24$, $\gamma = 0,02$ de la función objetivo DWA y el radio de inflado de 0.40 m, proporciona un punto de partida validado para experimentación.

Las limitaciones identificadas incluyen la dependencia de un mapeo de calidad para el éxito de las tareas posteriores (ADR-004), la ausencia de obstáculos dinámicos en la simulación y las restricciones cinemáticas propias del robot diferencial. Como trabajo futuro, se propone: (1) incorporar obstáculos dinámicos para validar la capacidad de evasión del DWA; (2) comparar el desempeño de Gmapping con Hector SLAM y Cartographer; (3) implementar un planificador local TEB para trayectorias más suaves en entornos con pasillos estrechos; y (4) migrar la simulación a ROS 2 y Gazebo Ignition para explorar las ventajas de la nueva arquitectura.

Referencias

Referencias

- Thrun, S., Burgard, W. and Fox, D. (2005) *Probabilistic Robotics*. Cambridge, MA: MIT Press.
- Siegwart, R., Nourbakhsh, I.R. and Scaramuzza, D. (2011) *Introduction to Autonomous Mobile Robots*. 2nd edn. Cambridge, MA: MIT Press.
- Grisetti, G., Stachniss, C. and Burgard, W. (2007) ‘Improved techniques for grid mapping with Rao-Blackwellized particle filters’, *IEEE Transactions on Robotics*, 23(1), pp. 34–46. doi:10.1109/TRO.2006.889486.
- Fox, D. (2003) ‘Adapting the sample size in particle filters through KLD-sampling’, *International Journal of Robotics Research*, 22(12), pp. 985–1003. doi:10.1177/0278364903022012001.
- Fox, D., Burgard, W. and Thrun, S. (1997) ‘The dynamic window approach to collision avoidance’, *IEEE Robotics & Automation Magazine*, 4(1), pp. 23–33. doi:10.1109/100.580977.
- Hart, P.E., Nilsson, N.J. and Raphael, B. (1968) ‘A formal basis for the heuristic determination of minimum cost paths’, *IEEE Transactions on Systems Science and Cybernetics*, 4(2), pp. 100–107. doi:10.1109/TSSC.1968.300136.
- Quigley, M., Conley, K., Gerkey, B., Faust, J., Foote, T., Leibs, J., Wheeler, R. and Ng, A.Y. (2009) ‘ROS: an open-source Robot Operating System’, in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA) Workshop on Open Source Software*. Kobe, Japan.
- Koenig, N. and Howard, A. (2004) ‘Design and use paradigms for Gazebo, an open-source multi-robot simulator’, in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. Sendai, Japan, pp. 2149–2154. doi:10.1109/IROS.2004.1389727.
- Montemerlo, M. and Thrun, S. (2007) ‘FastSLAM: a factored solution to the simultaneous localization and mapping problem’, in *Proceedings of the AAAI National Conference on Artificial Intelligence*. Vancouver, Canada, pp. 593–598.

-
- Marder-Eppstein, E., Berger, E., Foote, T., Gerkey, B. and Konolige, K. (2010) ‘The office marathon: Robust navigation in an indoor office environment’, in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. Anchorage, AK, pp. 300–307. doi:10.1109/ROBOT.2010.5509725.
- Rublee, E., Rabaud, V., Konolige, K. and Bradski, G. (2011) ‘ORB: an efficient alternative to SIFT or SURF’, in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. Barcelona, Spain, pp. 2564–2571. doi:10.1109/ICCV.2011.6126544.
- Lynch, K.M. and Park, F.C. (2017) *Modern Robotics: Mechanics, Planning, and Control*. Cambridge: Cambridge University Press.

Apéndice A: Fragmentos de Configuración

A.1 Comandos de Lanzamiento

Listing 4: Pipeline completo de lanzamiento por tarea.

```
1 # === Tarea 1: Mapeo ===
2 roslaunch robot_gazebo robot.launch
3 roslaunch robot_gazebo gmapping.launch
4 rosrun teleop_twist_keyboard teleop_twist_keyboard.py
5 rosrun map_server map_saver -f ~/maps/mapa_final
6
7 # === Tarea 2: Localización ===
8 roslaunch robot_gazebo robot.launch
9 rosrun map_server map_server ~/maps/mapa_final.yaml
10 roslaunch robot_gazebo amcl.launch
11 # (En RViz: 2D Pose Estimate)
12
13 # === Tarea 3: Navegación ===
14 roslaunch robot_gazebo robot.launch
15 rosrun map_server map_server ~/maps/mapa_final.yaml
16 roslaunch robot_gazebo amcl.launch
17 roslaunch robot_gazebo move_base.launch
18 # (En RViz: 2D Nav Goal)
```

A.2 Configuración YAML de Costmaps

Listing 5: Parámetros comunes de costmap (costmap_common_params.yaml).

```
1 obstacle_range: 3.5
2 raytrace_range: 4.0
3 inflation_radius: 0.40
4 cost_scaling_factor: 5.0
5
6 observation_sources: laser_scan_sensor
7
8 laser_scan_sensor:
9   {
```

```
10     sensor_frame: laser_frame,
11     data_type: LaserScan,
12     topic: /scan,
13     marking: true,
14     clearing: true,
15     expected_update_rate: 20.0,
16     min_obstacle_height: 0.0,
17     max_obstacle_height: 0.5,
18 }
```

A.3 Parámetros DWA

Listing 6: Planificador local DWA (dwa_local_planner_params.yaml).

```
1 DWAPlanerROS:
2   max_vel_x: 0.55
3   min_vel_x: -0.10
4   max_vel_theta: 1.0
5   min_in_place_vel_theta: 0.5
6
7   acc_lim_x: 2.5
8   acc_lim_theta: 3.2
9
10  sim_time: 1.5
11  vx_samples: 6
12  vtheta_samples: 20
13
14  path_distance_bias: 32.0
15  goal_distance_bias: 24.0
16  occdist_scale: 0.02
17
18  xy_goal_tolerance: 0.10
19  yaw_goal_tolerance: 0.05
20  controller_frequency: 10.0
```